

Vibe Coding Chat History - Lab 4: Endless Runner (PES1UG24CS703)

Session 01a0f034-8ed0-7646-b336-780fa077f0f3 · model opencode-go/deepseek-v4.1-flash · started 2026-09-30 02:46:11 UTC · exported 2026-09-30 08:44:34 · repository github.com/8figalltimepro/39_endless-runner

USER

Complete the assignment in @Lab_4_VibeCoding_Student_handout.pdf
The repo link given to me: https://github.com/SETAPESU26/39_endless-runner
SRN: PES1UG24CS703

No unnecessary python comments to be made. very simple and linear code. less verbose.

DO you have any screen recording tools with you? download if you need any.
Give me all the clean deliverables.

USER (ATTACHMENT)

 attached: Lab_4_VibeCoding_Student_handout.pdf (71355 bytes)

ASSISTANT

I'll start by reading the assignment handout and checking the repo.

```
→ read Lab_4_VibeCoding_Student_handout.pdf
→ bash ls -la; echo "----"; ls -la ~/screenshots 2>/dev/null; echo "----tools----"; which
ffmpeg screencapture osascript python3 git gh 2>/dev/null
```

ASSISTANT

```
→ bash cd /Users/sam/sem-5/se/Lab/vibe_coding && git clone
https://github.com/SETAPESU26/39_endless-runner.git 2>&1 | tail -5; echo "----"; ls -R
39_endless-runner | head -50; echo "----gh----"; gh auth status 2>&1 | head -10
→ bash ffmpeg -hide_banner -f avfoundation -list_devices true -i "" 2>&1 | head -30
```

ASSISTANT

Screen recording available (ffmpeg avfoundation `Capture screen 0`), gh authenticated as `8figalltimepro`. Reading the repo now.

```
→ read 39_endless-runner/README.md
→ bash cd /Users/sam/sem-5/se/Lab/vibe_coding/39_endless-runner && wc -l main.py
requirements.txt game/*.py && cat requirements.txt && echo "=== main.py ===" && cat
main.py
```

ASSISTANT

```
→ bash cd /Users/sam/sem-5/se/Lab/vibe_coding/39_endless-runner && for f in
game/game_engine.py game/player.py game/obstacle.py; do echo "=== $f ==="; cat -n "$f";
done
```

ASSISTANT

Repo understood: Pygame endless runner with 4 gaps (uncapped speed → tunneling, no game-over screen, no replay/difficulty, no sound). Checking toolchain before recording the "before" footage.

```
→ bash python3 -c "import pygame; print('pygame', pygame.version.ver)" 2>&1 | tail -3;
echo "---brew---"; ls /opt/homebrew/bin | grep -iE "pandoc|libre|sox|chrom" ; echo "---
apps---"; ls /Applications | grep -iE "chrome|libr...
→ bash cd /tmp && timeout 6 ffmpeg -hide_banner -loglevel error -f avfoundation -
framerate 30 -i "3:none" -t 2 -pix_fmt yuv420p -y /tmp/screentest.mp4 2>&1 | tail -5; ls
-la /tmp/screentest.mp4 2>/dev/null && ffprobe -v erro...
```

ASSISTANT

Screen capture works. Found `8figalltimepro/SE-Lab` — checking its convention.

```
→ bash cd /tmp && rm -rf selab_probe && git clone --depth 1
https://github.com/8figalltimepro/SE-Lab.git selab_probe 2>&1 | tail -2 && find
selab_probe -not -path "*/.git/*" -maxdepth 3 | sort | head -60
→ bash python3 --version; which -a python3 python3.12 python3.11 pip3; ls
/opt/homebrew/bin/python3* 2>/dev/null; echo "---venv test---"; python3 -m venv --help
>/dev/null 2>&1 && echo "venv ok"
```

ASSISTANT

Toolchain confirmed. Setting up work plan.

```
→ todo op=init, i=Planning lab deliverable work, list=[{'phase': 'Setup', 'items':
['Install pygame + verify original game runs', 'Determine screen/window layout for
recording']}, {'phase': 'Before video', 'items': ['Record ...
→ bash cd /Users/sam/sem-5/se/Lab/vibe_coding && python3 -m venv .venv && ./venv/bin/pip
install -q pygame 2>&1 | tail -3; ./venv/bin/python -c "import pygame; print('pygame',
pygame.version.ver)"
→ read xd://eval/computer
```

ASSISTANT

pygame 2.6.1 installed. Launching the original game to establish the recording layout.

```
→ bash cd /Users/sam/sem-5/se/Lab/vibe_coding/39_endless-runner && ../venv/bin/python
main.py
→ eval language=js, title=Desktop capabilities, code=const caps = await
computer.capabilities(); display(caps); const ds = await computer.displays();
display(ds);
```

ASSISTANT

```
→ eval code=const wins = await computer.windows({}); display(wins.filter(w =>
/runner|python|Terminal/i.test(w.app + " " + w.title)));, language=js, title=Finding
pygame window
```

ASSISTANT

```
→ eval code=const wins = await computer.windows({}); display(wins);, language=js,
title=Listing all windows
```

ASSISTANT

```
→ eval code=const shot = await computer.screenshot({ silent: false }); display(shot);,  
language=js, title=Desktop screenshot  
→ bash ffprobe -v error -select_streams v -show_entries  
stream=width,height,r_frame_rate,avg_frame_rate -of csv /tmp/screentest.mp4
```

ASSISTANT

Layout: 1440x900 @2x, game window at (320,222,800,428). Now setting up a visible terminal for the console-output evidence.

```
→ write /tmp/lab4/before_run.sh  
→ bash printf '' > proc://runner-before
```

ASSISTANT

```
→ bash printf '' > proc://runner-before/kill; sleep 1; pgrep -fl "main.py" | head
```

ASSISTANT

```
→ bash kill 96772; sleep 1; pgrep -fl "main.py" | head; echo "done"
```

ASSISTANT

```
→ bash open -a Terminal /tmp/lab4/before_run.sh; sleep 4; pgrep -fl "main.py" | head -3
```

USER

I have minimised all the other apps, now should be clean

ASSISTANT

```
→ eval code=const wins = await computer.windows({}); display(wins.map(w => ({app: w.app,  
title: w.title.slice(0,40), x: w.x, y: w.y, w: w.width, h: w.height, focused:  
w.focused}))); await computer.screenshot();, language=js,...
```

ASSISTANT

```
→ bash pgrep -fl "main.py" | head -3; echo "---"; ps aux | grep -c "[b]efore_run.sh"  
→ eval code=const win = await computer.window({ app: "Terminal" }); display({id: win.id,  
title: win.title, bounds: win.bounds}); const tree = await win.ax({ maxDepth: 3 });  
display(tree);, language=js, title=Inspecting Termi...
```

ASSISTANT

Game launched (pid 97662). Verifying window placement.

```
→ eval code=const wins = await computer.windows({}); display(wins.filter(w =>  
/Python|Terminal/.test(w.app)).map(w => ({app: w.app, x: w.x, y: w.y, w: w.width, h:  
w.height, focused: w.focused}))); await computer.screenshot()...
```

ASSISTANT

```
→ eval code=const wins = await computer.windows({}); display(wins.map(w => ({app: w.app,  
title: w.title.slice(0,36), x: w.x, y: w.y, w: w.width, h: w.height, focused:  
w.focused})));, language=js, title=Full window list
```

USER

Better give me the instructions and what to do so that i can do that. instead of u mkaing it difficult

ASSISTANT

Understood — I'll hand you the manual bits as one-liners and do everything else myself. Snapshotting the pristine code first (so the "before" recording never depends on my edits).

```
→ bash cd /Users/sam/sem-5/se/Lab/vibe_coding && kill 97662 2>/dev/null; osascript -e 'tell application "Terminal" to close (every window whose name contains "before_run")' 2>/dev/null; rm -rf before_snapshot && cp -R 39_end...  
→ bash cd /tmp && timeout 8 ffmpeg -hide_banner -loglevel error -f avfoundation -framerate 30 -i "3:0" -t 2 -y /tmp/mictest.mp4 2>&1 | tail -3; ls -la /tmp/mictest.mp4 2>/dev/null && ffprobe -v error -show_entries stream=cod...
```

ASSISTANT

```
→ bash cd /tmp && timeout 8 ffmpeg -hide_banner -f avfoundation -framerate 30 -i "3:0" -t 2 -y /tmp/mictest.mp4 2>&1 | grep -iE "error|denied|permission|not authorized|audio" | head -8; echo "exit=$?"; ls -la /tmp/mictest.mp...
```

USER

i will use normal mac record, please dont overthink. just give me straight instructions and i will start the recording

ASSISTANT

Fine — mic capture is blocked, so screen recording is yours. Writing the demo harness now, then straight instructions.

```
→ write /Users/sam/sem-5/se/Lab/vibe_coding/before_snapshot/before_tunnel_demo.py
```

ASSISTANT

```
→ bash cd /Users/sam/sem-5/se/Lab/vibe_coding/before_snapshot && SDL_VIDEODRIVER=dummy  
../.venv/bin/python -c " import pygame; pygame.init(); pygame.display.set_mode((800,400))  
from game.game_engine import GameEngine e = Gam...
```

ASSISTANT

Tunneling repro confirmed headlessly: at speed 75 px/frame, 6 obstacles passed through the player, `game\_over=False`.

Record the BEFORE video now (2 clips, ~16 s total)

Use **Cmd+Shift+5** → Record Entire Screen. Leave both files on the Desktop — no renaming needed.

Clip 1 — real gameplay (broken: game over only prints to console):

```
bash
cd /Users/sam/sem-5/se/Lab/vibe_coding/before_snapshot
SDL_VIDEO_WINDOW_POS=320,222 ../../venv/bin/python main.py
```

- Keep that Terminal window **visible** (the console print is the evidence).
- Start recording → click the game window once → press `Space` 3-4 times (jump) → then stop jumping and let a green block hit you.
- Screen freezes, console prints `Game over! Final score: 0` → no game-over screen, no restart. Wait 3 s. Stop. (~10 s)

Clip 2 — tunneling bug (Task 1), ~6 s:

```
bash
cd /Users/sam/sem-5/se/Lab/vibe_coding/before_snapshot
../../venv/bin/python before_tunnel_demo.py
```

- Record ~6 s. Green blocks jump straight over the white player, nothing registers a hit, score climbs, no game over. Stop. (~6 s)

Tell me when both are on the Desktop — I'll be implementing all 4 fixes meanwhile.

USER

I have uploaded the entire recording showcasing the 2 cases in @before_snapshot/Before_Screens_recording.mov
Convert the .mov to .mp4 first.

Then make the solution and then give me the instructions for the after_video

USER (ATTACHMENT)

 attached: before_snapshot/Before_Screens_recording.mov (None bytes)

ASSISTANT

Converting and analyzing the recording.

```
bash cd /Users/sam/sem-5/se/Lab/vibe_coding/before_snapshot && ffmpeg -hide_banner -loglevel error -i Before_Screens_recording.mov -c:v libx264 -preset medium -crf 20 -pix_fmt yuv420p -an -y /tmp/lab4/before_full.mp4 && ff...
```

ASSISTANT

```
→ bash mkdir -p /tmp/lab4/sheet && cd /tmp/lab4/sheet && rm -f *.png && ffmpeg -hide_banner -loglevel error -i /tmp/lab4/before_full.mp4 -vf "fps=1/3,scale=640:-2,drawtext=text='%{pts\:hms}':x=6:y=6:fontsize=28:fontcolor=yel...
```

ASSISTANT

```
→ read /tmp/lab4/sheet/sheet1.png
```

ASSISTANT

```
→ bash cd /tmp/lab4 && rm -rf f && mkdir f && for t in 16 30 40; do ffmpeg -hide_banner -loglevel error -ss $t -i before_full.mp4 -frames:v 1 -y f/t$t.png; done; ls -la f/
```

ASSISTANT

```
→ read /tmp/lab4/f/t16.png  
→ read /tmp/lab4/f/t30.png
```

ASSISTANT

Key evidence located: score 6 + `Game over! Final score: 6` in console (no game-over UI), and the slow-mo tunneling run. Pinning exact cut points.

```
→ bash cd /tmp/lab4 && for t in 17.2 32.0; do ffmpeg -hide_banner -loglevel error -ss $t -i before_full.mp4 -frames:v 1 -y f/t$t.png; done && ffmpeg -hide_banner -loglevel error -i f/t17.2.png -vf "crop=1600:860:0:60" -y f/c...
```

ASSISTANT

```
→ read /tmp/lab4/f/c17.png  
→ read /tmp/lab4/f/c32.png
```

ASSISTANT

Timeline decoded: game runs 0:06–0:17 (collision → frozen screen, console prints `Game over! Final score: 6`), tunnel demo 0:26–0:33. Building the 10 s before-video.

```
→ bash cd /tmp/lab4 && ffmpeg -hide_banner -loglevel error -i before_full.mp4 -filter_complex "[0:v]trim=13.5:19.5,setpts=PTS-STARTPTS,crop=1600:860:0:60[a];[0:v]trim=28:32,setpts=PTS-STARTPTS,crop=1600:860:0:60[b];[a][b]con...
```

ASSISTANT

```
→ bash cd /tmp/lab4 && ffmpeg -hide_banner -loglevel error -i before_full.mp4 -filter_complex "[0:v]trim=13.6:19.6,setpts=PTS-STARTPTS,crop=1600:860:0:60[a];[0:v]trim=28:32.2,setpts=PTS-STARTPTS,crop=1600:860:0:60[b];[a][b]c...
```

ASSISTANT

```
→ read /tmp/lab4/sheet/bsheet.png
```

ASSISTANT

```
→ bash cd /tmp/lab4 && ffmpeg -hide_banner -loglevel error -i before_full.mp4 -filter_complex "[0:v]trim=13:18.5,setpts=PTS-STARTPTS,crop=1600:860:0:60[a];[0:v]trim=28:32.5,setpts=PTS-STARTPTS,crop=1600:860:0:60[b];[a][b]con...
```

ASSISTANT

Before-video is cut (10.2 s, 1600x860). Now implementing all four tasks.

```
→ write 39_endless-runner/game/sound.py
→ write 39_endless-runner/game/obstacle.py
→ write 39_endless-runner/game/player.py
```

ASSISTANT

```
→ write 39_endless-runner/game/game_engine.py
```

ASSISTANT

```
→ edit 39_endless-runner/game/game_engine.py
→ edit 39_endless-runner/game/game_engine.py
```

ASSISTANT

```
→ write 39_endless-runner/main.py
```

ASSISTANT

```
→ bash cd /Users/sam/sem-5/se/Lab/vibe_coding/39_endless-runner && SDL_VIDEODRIVER=dummy
SDL_AUDIODRIVER=dummy ../.venv/bin/python -c " import pygame pygame.mixer.pre_init(44100,
-16, 1, 512) pygame.init(); pygame.display.se...
```

ASSISTANT

Logic verified: swept collision catches the 75 px/frame skip that the plain rect test missed; cap holds at 22; replay/difficulty/quit all work. Now proving audio actually plays.

```
→ bash mkdir -p /tmp/lab4/audiotest && cd /tmp/lab4/audiotest && rm -f sdlaudio.raw &&
SDL_VIDEODRIVER=dummy SDL_AUDIODRIVER=disk SDL_DISKAUDIOFILE=sdlaudio.raw /Users/sam/sem-
5/se/Lab/vibe_coding/.venv/bin/python -c " impor...
```

ASSISTANT

Audio confirmed (19.8k non-silent samples, peak 40%). Verifying the new screens render correctly.

```
→ bash cd /Users/sam/sem-5/se/Lab/vibe_coding/39_endless-runner && SDL_VIDEODRIVER=dummy
SDL_AUDIODRIVER=dummy ../.venv/bin/python -c " import pygame pygame.mixer.pre_init(44100,
-16, 1, 512) pygame.init() screen = pygame.di...
```

ASSISTANT

```
→ read /tmp/lab4/f/ui_menu.png
→ read /tmp/lab4/f/ui_gameover.png
```

ASSISTANT

UI verified. Updating README, then a real-window smoke test.

```
→ edit 39_endless-runner/README.md
```

ASSISTANT

```
→ bash cd /Users/sam/sem-5/se/Lab/vibe_coding/39_endless-runner && grep -n "^#" README.md && python3 -c "print(open('README.md').read().count('| Easy | 5 px/frame | 95 frames |'))"
→ bash cd /Users/sam/sem-5/se/Lab/vibe_coding/39_endless-runner && SDL_VIDEO_WINDOW_POS=320,222 ../../venv/bin/python main.py
```

USER

dw the vid will be recorded on OBS by me so the audio will also be available.

ASSISTANT

OBS for the after-video — perfect. Smoke-testing the real window now.

```
→ eval code=const win = await computer.window({ title: "Endless Runner - Pygame Version" }); display({id: win.id, bounds: win.bounds, focused: win.focused}); await win.screenshot(); await win.press("2"); await new Promise(r ...
```

ASSISTANT

Real-window path works. Confirming game-over + replay in the real app, then I'll stop it.

```
→ eval code=const win = await computer.window({ title: "Endless Runner - Pygame Version" }); await new Promise(r => setTimeout(r, 3500)); await win.press("space"); await new Promise(r => setTimeout(r, 900)); await win.screen...
```

ASSISTANT

```
→ bash kill 7037 2>/dev/null; sleep 0.5; pgrep -fl "main.py" | head -2; echo "game stopped"
→ todo i=Updating progress, items=['Install pygame + verify original game runs', 'Determine screen/window layout for recording', 'Record 10s BEFORE video (console-only game over + uncapped-speed tunneling)', 'Task 1: swept c...
```

ASSISTANT

Correcting the tracker (my `done` call over-marked) and looking for the session transcript to export.

```
→ todo i=Resetting tracker to真实 remaining work, items=['Cut 10s AFTER video from OBS recording (with audio)', 'Export full chat history as PDF', 'Assemble Lab-4 folder (videos, code, chat history)', 'Push to personal repo 39...
→ bash ls -d ~/.omp ~/Library/Application\ Support/omp ~/.local/share/omp 2>/dev/null; find ~/.omp ~/Library/Application\ Support/omp -maxdepth 3 -name "*.json*" -newermt "-6 hours" 2>/dev/null | head -20
```

ASSISTANT

```
→ bash find /Users/sam/.omp -maxdepth 4 -type d | head -20; echo "---- recent files ----"; find /Users/sam/.omp -type f -newermt "-3 hours" 2>/dev/null | head -20
```

ASSISTANT

```
→ read history://
→ bash ls ~/Library/Application\ Support/ 2>/dev/null | grep -i -E "omp|opcode|ocd" ; find ~/Library/Application\ Support -maxdepth 2 -iname "*omp*" 2>/dev/null | head; find /Users/sam/.omp /Users/sam/Library -maxdepth 5 -...
```

ASSISTANT

Found the session store. Checking both raw session files and the transcript API.

```
→ bash ls -lt /Users/sam/.omp/agent/sessions/ | head -10; echo "=== vibe_coding ==="; ls -la /Users/sam/.omp/agent/sessions/ | grep -i "vibe" ; SESS=$(ls -dt /Users/sam/.omp/agent/sessions/*vibe*/ 2>/dev/null | head -1); echo...
```

ASSISTANT

```
→ bash cd /Users/sam/.omp/agent/sessions/-sem-5-se-Lab-vibe_coding && F=2026-09-30T02-46-11-408Z_01a0f034-8ed0-7646-b336-780fa077f0f3.jsonl && wc -l $F && python3 -c " import json rows=[json.loads(l) for l in open('$F') if l..."
```

ASSISTANT

```
→ bash cd /Users/sam/.omp/agent/sessions/-sem-5-se-Lab-vibe_coding && python3 -c " import json rows=[json.loads(l) for l in open('2026-09-30T02-46-11-408Z_01a0f034-8ed0-7646-b336-780fa077f0f3.jsonl') if l.strip()] msgs=[r fo..."
```

ASSISTANT

```
→ bash cd /Users/sam/.omp/agent/sessions/-sem-5-se-Lab-vibe_coding && python3 -c " import json rows=[json.loads(l) for l in open('2026-09-30T02-46-11-408Z_01a0f034-8ed0-7646-b336-780fa077f0f3.jsonl') if l.strip()] msgs=[r['m..."
```

ASSISTANT

```
→ bash cd /Users/sam/.omp/agent/sessions/-sem-5-se-Lab-vibe_coding && python3 -c " import json rows=[json.loads(l) for l in open('2026-09-30T02-46-11-408Z_01a0f034-8ed0-7646-b336-780fa077f0f3.jsonl') if l.strip()] for t in [..."
```

ASSISTANT

Session transcript is fully captured. Writing the chat-history PDF generator.

```
→ write /tmp/lab4/make_chat_history.py
```

ASSISTANT

```
→ bash cd /tmp/lab4 && python3 make_chat_history.py /tmp/lab4 2>&1 | tail -3 && python3 -c " import json rows=[json.loads(l) for l in open('/Users/sam/.omp/agent/sessions/-sem-5-se-Lab-vibe_coding/2026-09-30T02-46-11-408Z_01..."
```